BBCo Orders Pipeline — Submission

Links:
- Google sheet with the data

Task extension:
- Google BigQuery
  - Shared BQ data view access with Rukshika and Vikshaka
- GitHub repo
  - Public repo (should be visible)

Overview

End-to-end data pipeline in Google Apps Script. Single entry point `runPipeline()` runs cleaning → anomaly detection → summary tables → Slack notification. A separate `runMonitoringChecks()` function implements production monitoring.

Beyond the assessment scope, I also built out a more automated production-style version with the following architecture:

Google Sheet tab → external table in BigQuery `raw_data` dataset → dbt cleaning into `ods` dataset → datamart layer with aggregations and filters → loaded back to a Google Sheet tab `items_by_country_channel_month` → dynamic dashboard sourcing from that tab.

For orchestration I went with the simplest viable option - a GitHub Actions workflow scheduled to run every 3 hours.

Happy to walk through this on the call.

Section 1a — Cleaning (`Orders_Cleaned`)

Reads `Orders_Raw` in one batch, deduplicates by `order_id` using a `Set` (keeps first occurrence), then normalizes:

- `financial_status` → lowercase, trimmed
- `currency`, `country_code` → uppercase, trimmed
- `created_at` → parsed via three explicit regexes (ISO, `YYYY/MM/DD`, `DD/MM/YYYY HH:MM`) and reformatted to `YYYY-MM-DD HH:MM:SS`
- Adds `data_quality_flag = 'INCOMPLETE'` when `financial_status` or `source_channel` is null

Section 1b — Anomaly Detection (`QC_Flags`)

Three checks against `Orders_Cleaned`: revenue math reconciliation (`net_revenue ≠ total_price - total_discounts`, ±$0.01 tolerance), `total_price ≤ 0`, `quantity ≤ 0`. Each flag captures `order_id`, reason, actual, expected.

Pattern observed: seeded anomalies cluster in math reconciliation — e.g. `ORD-28909` has `net_revenue` ($167.90) higher than `total_price` ($132.39), suggesting a discount-sign inversion or refund miscoding upstream.

Section 1c — Summary Tables (`Summary`)

Filters out `INCOMPLETE` rows, then builds three stacked tables: Revenue by Channel, Orders by Financial Status (with % of total), and Top Products sorted by `total_revenue` descending. All money rounded to 2dp.

Section 1d — Slack Pipeline Summary

Posts to webhook with: orders processed, duplicates removed, paid-only gross revenue, top channel by net revenue, anomaly count, pass rate `(cleaned - INCOMPLETE) / total × 100%`. Status is `PASSED` if anomalies = 0 AND pass rate ≥ 95%, else `FAILED`.

Section 1e — Monitoring (`runMonitoringChecks`)

Separate function so it can run on its own schedule. Four checks:

Check	Threshold	Severity
Order count variance	Today's count >20% off daily avg	WARNING
Zero/negative paid revenue	Any `paid` order with `total_price ≤ 0`	CRITICAL
Pass rate drop	Below 90%	CRITICAL
Duplicate rate	Above 5% of raw rows	WARNING

Alerts batched into one Slack message. Silent on success. "Today" simulated as the latest date in the dataset.

Setup

1. Import `BBCo_Orders_Sample.xlsx` into a sheet named `Orders_Raw`.
2. Extensions → Apps Script → paste the script.
3. Create a Slack app at api.slack.com/apps, enable Incoming Webhooks, add to `#pipeline-alerts`, copy URL into `CONFIG.SLACK_WEBHOOK`.

4. Run `runPipeline`, authorize, then run `runMonitoringChecks`.

Section 1f — Decisions

Trickiest cleaning challenge. The tasks were largely straightforward, but if I had to pick one it would be the date normalization. The source had three incompatible formats including `DD/MM/YYYY`, which JavaScript's `new Date()` silently misparses as US `MM/DD/YYYY` — swapping day and month with no error thrown. I used explicit regex matchers per format with native parsing as a last-resort fallback, and left unparseable values untouched so they remain findable rather than silently corrupted.

Scaling to 500k rows. Single `getValues()` would hit the 50MB memory limit and 6-minute runtime cap. I'd batch-read in ~10k-row chunks with `PropertiesService` checkpointing for resumability. At that scale Apps Script isn't the right tool — the cleaning logic belongs in BigQuery SQL or a dbt model, with Apps Script reduced to orchestration.

Thresholds. Anchored to operational impact, not round numbers. Pass rate <90% and zero-revenue paid orders are `CRITICAL` because they break downstream reporting or indicate ETL bugs that compound silently. Order count variance and duplicate rate are `WARNING` because both have legitimate explanations (campaigns, retries) and need a human eye, not a 3 AM page. The 95% gate in 1d is stricter than the 90% gate in monitoring on purpose: the pipeline should fail loudly during a run; monitoring should only trip when conditions genuinely degrade.

Full Apps Script Code

```
/**
* BBCo Orders Pipeline
* Entry point: runPipeline()
* Monitoring: runMonitoringChecks()
*/

const CONFIG = {
 RAW_SHEET:      'Orders_Raw',
 CLEAN_SHEET:    'Orders_Cleaned',
 QC_SHEET:       'QC_Flags',
 SUMMARY_SHEET:  'Summary',
 SLACK_WEBHOOK:
'https://hooks.slack.com/services/T0B0X8G0Q4V/B0B0U9FMTFD/kelGExVHMxy6KEbbl
30yIa7u',
 TOLERANCE:      0.01,
};
```

```
/* ============================================================
* SECTION 1 — MAIN PIPELINE
* ============================================================ */
function runPipeline() {
 const ss = SpreadsheetApp.getActiveSpreadsheet();

 const cleanResult   = cleanOrders_(ss);             // 1a
 const qcResult      = detectAnomalies_(ss);         // 1b
 const summaryResult = buildSummary_(ss);            // 1c
 postPipelineSummary_(cleanResult, qcResult, summaryResult); // 1d

 ss.toast('Pipeline complete — see Slack for summary', 'runPipeline', 5);
}

/* ---------- 1a. Cleaning ---------- */
function cleanOrders_(ss) {
 const src = ss.getSheetByName(CONFIG.RAW_SHEET);
 if (!src) throw new Error(`Sheet "${CONFIG.RAW_SHEET}" not found`);

 const data = src.getDataRange().getValues();
 const headers = data[0];
 const rows = data.slice(1);

 const idx = indexHeaders_(headers, [

'order_id','created_at','customer_id','source_channel','financial_status',
   'fulfillment_status','total_price','total_discounts','net_revenue',
   'currency','product_title','quantity','country_code'
 ]);

 // Deduplicate by order_id, keep first
 const seen = new Set();
 let duplicates = 0;
 const deduped = [];
```

```
for (const row of rows) {
  const id = String(row[idx.order_id]).trim();
  if (!id) continue;
  if (seen.has(id)) { duplicates++; continue; }
  seen.add(id);
  deduped.push(row);
}

// Clean fields
let incompleteCount = 0;
const cleaned = deduped.map(row => {
  const r = row.slice();

  r[idx.financial_status] = lower_(r[idx.financial_status]);
  r[idx.currency]         = upper_(r[idx.currency]);
  r[idx.country_code]     = upper_(r[idx.country_code]);

  const d = parseDate_(r[idx.created_at]);
  if (d) r[idx.created_at] = formatDate_(d);

  const fsEmpty = !r[idx.financial_status];
  const scEmpty = !String(r[idx.source_channel]).trim();
  const flag = (fsEmpty || scEmpty) ? 'INCOMPLETE' : '';
  if (flag) incompleteCount++;

  r.push(flag);
  return r;
});

// Write Orders_Cleaned
const out = ensureSheet_(ss, CONFIG.CLEAN_SHEET);
const newHeaders = headers.concat(['data_quality_flag']);
out.getRange(1, 1, 1,
newHeaders.length).setValues([newHeaders]).setFontWeight('bold');
if (cleaned.length) {
```

```
  out.getRange(2, 1, cleaned.length,
newHeaders.length).setValues(cleaned);
 }
 out.setFrozenRows(1);

 Logger.log(`Cleaning: ${rows.length} in → ${cleaned.length} out,
${duplicates} duplicates removed, ${incompleteCount} INCOMPLETE`);
 return { totalIn: rows.length, duplicates, cleanedRows: cleaned.length,
incompleteCount };
}

/* ---------- 1b. Anomaly Detection ---------- */
function detectAnomalies_(ss) {
 const src = ss.getSheetByName(CONFIG.CLEAN_SHEET);
 const data = src.getDataRange().getValues();
 const headers = data[0];
 const rows = data.slice(1);
 const idx = indexHeaders_(headers,
['order_id','total_price','total_discounts','net_revenue','quantity','finan
cial_status']);

 const flags = [];
 const patterns = { math_mismatch: 0, nonpositive_price: 0,
nonpositive_qty: 0 };

 for (const row of rows) {
   const id    = row[idx.order_id];
   const price = Number(row[idx.total_price]);
   const disc  = Number(row[idx.total_discounts]);
   const net   = Number(row[idx.net_revenue]);
   const qty   = Number(row[idx.quantity]);

   if (!isNaN(price) && !isNaN(disc) && !isNaN(net)) {
     const expected = round2_(price - disc);
     if (Math.abs(net - expected) > CONFIG.TOLERANCE) {
```

```
        flags.push([id, 'net_revenue mismatch', net, expected]);
        patterns.math_mismatch++;
      }
    }
    if (!isNaN(price) && price <= 0) {
      flags.push([id, 'total_price <= 0', price, '> 0']);
      patterns.nonpositive_price++;
    }
    if (!isNaN(qty) && qty <= 0) {
      flags.push([id, 'quantity <= 0', qty, '> 0']);
      patterns.nonpositive_qty++;
    }
  }

  const out = ensureSheet_(ss, CONFIG.QC_SHEET);
  const qcHeaders =
['order_id','flag_reason','actual_value','expected_value'];
  out.getRange(1, 1, 1,
qcHeaders.length).setValues([qcHeaders]).setFontWeight('bold');
  if (flags.length) {
    out.getRange(2, 1, flags.length, qcHeaders.length).setValues(flags);
  }
  out.setFrozenRows(1);

  Logger.log(`QC: ${flags.length} anomalies — ${JSON.stringify(patterns)}`);
  return { totalFlags: flags.length, patterns, totalRows: rows.length };
}

/* ---------- 1c. Summary Tables ---------- */
function buildSummary_(ss) {
  const src = ss.getSheetByName(CONFIG.CLEAN_SHEET);
  const data = src.getDataRange().getValues();
  const headers = data[0];
  const idx = indexHeaders_(headers, [
    'source_channel','financial_status','total_price','total_discounts',
```

```
  'net_revenue','quantity','product_title','data_quality_flag'
 ]);

 // Exclude INCOMPLETE rows for all summaries
 const rows = data.slice(1).filter(r => r[idx.data_quality_flag] !==
'INCOMPLETE');

 // Table 1 — Revenue by Channel
 const byChannel = {};
 for (const r of rows) {
   const ch = r[idx.source_channel] || '(unknown)';
   const t = byChannel[ch] || (byChannel[ch] = {orders:0, gross:0,
discounts:0, net:0});
   t.orders++;
   t.gross     += Number(r[idx.total_price])     || 0;
   t.discounts += Number(r[idx.total_discounts]) || 0;
   t.net       += Number(r[idx.net_revenue])     || 0;
 }
 const t1 =
[['source_channel','total_orders','gross_revenue','total_discounts','net_re
venue','avg_order_value']];
 for (const [ch, t] of Object.entries(byChannel)) {
   t1.push([ch, t.orders, round2_(t.gross), round2_(t.discounts),
round2_(t.net), round2_(t.gross/t.orders)]);
 }

 // Table 2 — Orders by Financial Status
 const byStatus = {};
 for (const r of rows) {
   const s = r[idx.financial_status] || '(unknown)';
   const t = byStatus[s] || (byStatus[s] = {orders:0, gross:0});
   t.orders++;
   t.gross += Number(r[idx.total_price]) || 0;
 }
 const totalOrders = rows.length;
```

```
const t2 = [['financial_status','order_count','gross_revenue','% of total
orders']];
for (const [s, t] of Object.entries(byStatus)) {
  t2.push([s, t.orders, round2_(t.gross),
round2_((t.orders/totalOrders)*100) + '%']);
}

// Table 3 — Top Products (sorted by total_revenue desc)
const byProduct = {};
for (const r of rows) {
  const p = r[idx.product_title] || '(unknown)';
  const t = byProduct[p] || (byProduct[p] = {units:0, revenue:0});
  t.units   += Number(r[idx.quantity])    || 0;
  t.revenue += Number(r[idx.total_price]) || 0;
}
const t3rows = Object.entries(byProduct).map(([p,t]) =>
  [p, t.units, round2_(t.revenue), t.units ? round2_(t.revenue/t.units) :
0]
).sort((a,b) => b[2] - a[2]);
const t3 =
[['product_title','total_units_sold','total_revenue','avg_price'],
...t3rows];

// Write all three tables stacked
const out = ensureSheet_(ss, CONFIG.SUMMARY_SHEET);
let row = 1;
row = writeBlock_(out, row, 'Table 1 — Revenue by Channel', t1) + 2;
row = writeBlock_(out, row, 'Table 2 — Orders by Financial Status', t2) +
2;
writeBlock_(out, row, 'Table 3 — Top Products', t3);

// Find top channel by net revenue for Slack summary
const topChannel = t1.slice(1).sort((a,b) => b[4]-a[4])[0] || ['(none)',
0, 0, 0, 0];
```

```
 // Gross revenue from PAID orders only (1d requirement)
 const paidGross = rows
   .filter(r => r[idx.financial_status] === 'paid')
   .reduce((s, r) => s + (Number(r[idx.total_price]) || 0), 0);

 return {
   paidGross: round2_(paidGross),
   topChannel: { name: topChannel[0], net_revenue: topChannel[4] },
   totalRowsExclIncomplete: totalOrders,
 };
}

/* ---------- 1d. Slack Pipeline Summary ---------- */
function postPipelineSummary_(cleanResult, qcResult, summaryResult) {
 const passRate = round2_(((cleanResult.cleanedRows -
cleanResult.incompleteCount) / cleanResult.totalIn) * 100);
 const status = (qcResult.totalFlags > 0 || passRate < 95) ? 'FAILED' :
'PASSED';
 const emoji = status === 'PASSED' ? ':white_check_mark:' : ':x:';

 const text =
   `${emoji} *Pipeline ${status}*\n` +
   `• Orders processed: *${cleanResult.totalIn}* (duplicates removed:
*${cleanResult.duplicates}*)\n` +
   `• Gross revenue (paid only): *$${summaryResult.paidGross}*\n` +
   `• Top channel by net revenue: *${summaryResult.topChannel.name}*
($${summaryResult.topChannel.net_revenue})\n` +
   `• QC anomalies: *${qcResult.totalFlags}*\n` +
   `• Data quality pass rate: *${passRate}%*`;

 postToSlack_(text);
}

/* ==========================================================
* SECTION 1e — MONITORING CHECKS
```

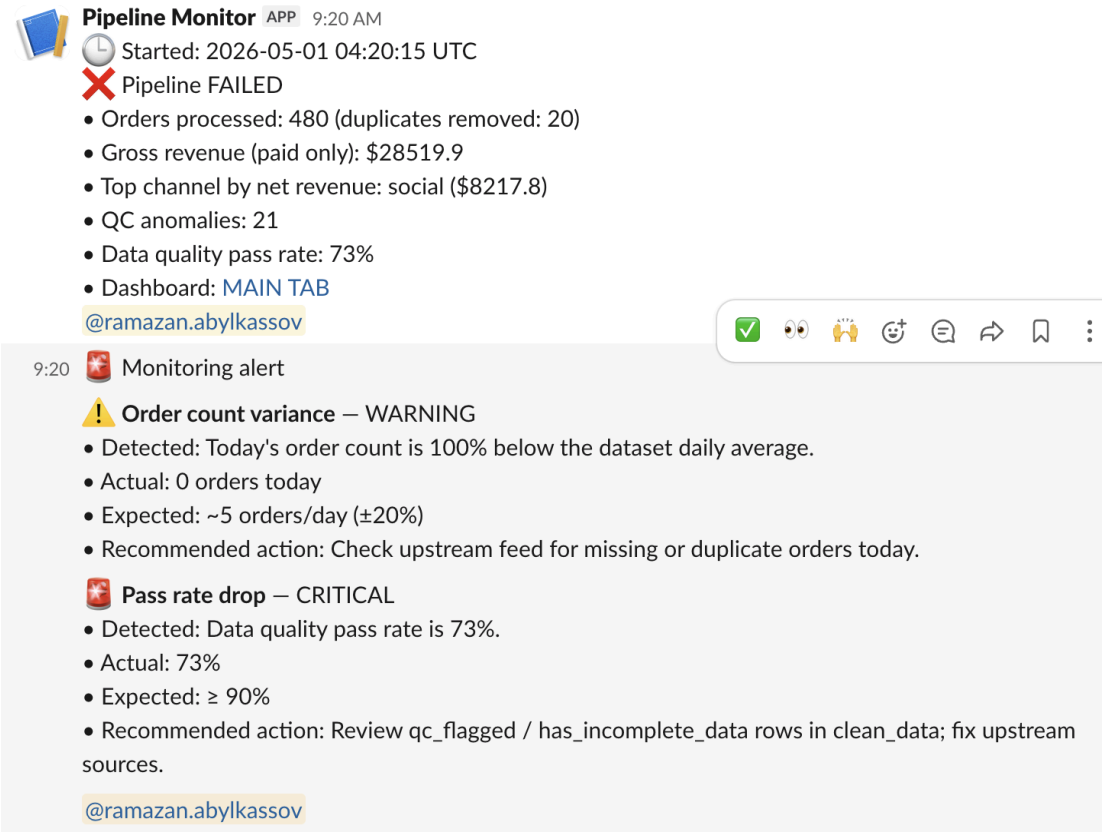
```
* ========================================================== */
function runMonitoringChecks() {
 const ss = SpreadsheetApp.getActiveSpreadsheet();
 const clean = ss.getSheetByName(CONFIG.CLEAN_SHEET);
 if (!clean) throw new Error('Run runPipeline() first — Orders_Cleaned not
found');

 const data = clean.getDataRange().getValues();
 const headers = data[0];
 const rows = data.slice(1);
 const idx = indexHeaders_(headers,
['created_at','total_price','financial_status','data_quality_flag','order_i
d']);

 const alerts = [];

 // ---- Check 1: Order count variance (today vs dataset daily average)
----
 const dailyCounts = {};
 for (const r of rows) {
   const dateStr = String(r[idx.created_at]).substring(0, 10); //
YYYY-MM-DD
   if (!dateStr) continue;
   dailyCounts[dateStr] = (dailyCounts[dateStr] || 0) + 1;
 }
 const counts = Object.values(dailyCounts);
 const avgDaily = counts.reduce((s,c) => s+c, 0) / Math.max(counts.length,
1);
 // "Today" simulated as the most recent date in the dataset
 const sortedDates = Object.keys(dailyCounts).sort();
 const todayKey = sortedDates[sortedDates.length - 1];
 const todayCount = dailyCounts[todayKey] || 0;
 const variancePct = avgDaily ? ((todayCount - avgDaily) / avgDaily) * 100
: 0;
 if (Math.abs(variancePct) > 20) {
```

```
  alerts.push({
    check: 'Order Count Variance',
    severity: 'WARNING',
    detected: `${todayKey} had ${todayCount} orders
(${variancePct.toFixed(1)}% vs avg)`,
    actual: todayCount,
    expected: `~${avgDaily.toFixed(1)} (±20%)`,
    action: 'Investigate traffic source, ad spend, or platform issue for
this date',
  });
}

// ---- Check 2: Zero/negative revenue on paid orders ----
const badPaid = rows.filter(r =>
  r[idx.financial_status] === 'paid' && Number(r[idx.total_price]) <= 0
);
if (badPaid.length > 0) {
  alerts.push({
    check: 'Zero/Negative Revenue (Paid)',
    severity: 'CRITICAL',
    detected: `${badPaid.length} paid orders with total_price <= 0`,
    actual: badPaid.length,
    expected: 0,
    action: 'Audit upstream order capture — likely ETL bug or refund
miscoding',
  });
}

// ---- Check 3: Pass rate drop ----
const incomplete = rows.filter(r => r[idx.data_quality_flag] ===
'INCOMPLETE').length;
const passRate = ((rows.length - incomplete) / rows.length) * 100;
if (passRate < 90) {
  alerts.push({
    check: 'Pass Rate Drop',
```

```
    severity: 'CRITICAL',
    detected: `Pass rate ${passRate.toFixed(1)}%`,
    actual: `${passRate.toFixed(1)}%`,
    expected: '>= 90%',
    action: 'Investigate source_channel/financial_status capture
upstream',
  });
}

// ---- Check 4: Duplicate rate ----
// Re-derive from raw vs cleaned counts (cleaning already deduped, so we
recompute)
const raw = ss.getSheetByName(CONFIG.RAW_SHEET);
const rawRows = raw.getLastRow() - 1;
const dupRate = ((rawRows - rows.length) / rawRows) * 100;
if (dupRate > 5) {
  alerts.push({
    check: 'Duplicate Rate',
    severity: 'WARNING',
    detected: `${dupRate.toFixed(1)}% of raw rows were duplicates`,
    actual: `${dupRate.toFixed(1)}%`,
    expected: '<= 5%',
    action: 'Check ETL idempotency — duplicate ingestion likely',
  });
}

if (alerts.length === 0) {
  Logger.log('All monitoring checks passed.');
  return;
}

// Build Slack alert message
const blocks = alerts.map(a =>
  `*${a.severity === 'CRITICAL' ? ':rotating_light:' : ':warning:'}
${a.check}* (${a.severity})\n` +
```

```
    `> Detected: ${a.detected}\n` +
    `> Actual: \`${a.actual}\` | Expected: \`${a.expected}\`\n` +
    `> Action: ${a.action}`
  ).join('\n\n');
  postToSlack_(`*Monitoring Alert — ${alerts.length} check(s) failed*\n\n${blocks}`);
  Logger.log(`Monitoring posted ${alerts.length} alert(s)`);
}

/* ============================================================
* HELPERS
* ============================================================ */
function lower_(v) { return v ? String(v).trim().toLowerCase() : ''; }
function upper_(v) { return v ? String(v).trim().toUpperCase() : ''; }
function round2_(n) { return Math.round(n * 100) / 100; }

function indexHeaders_(headers, names) {
  const out = {};
  for (const n of names) {
    const i = headers.indexOf(n);
    if (i === -1) throw new Error(`Missing column: ${n}`);
    out[n] = i;
  }
  return out;
}

function ensureSheet_(ss, name) {
  let s = ss.getSheetByName(name);
  if (s) s.clear();
  else s = ss.insertSheet(name);
  return s;
}

function writeBlock_(sheet, startRow, title, table) {
```

```
 sheet.getRange(startRow,
1).setValue(title).setFontWeight('bold').setFontSize(12);
 sheet.getRange(startRow + 1, 1, table.length,
table[0].length).setValues(table);
 sheet.getRange(startRow + 1, 1, 1, table[0].length).setFontWeight('bold');
 return startRow + 1 + table.length;
}

function parseDate_(v) {
 if (!v && v !== 0) return null;
 if (v instanceof Date && !isNaN(v)) return v;
 const s = String(v).trim();
 if (!s) return null;

 let m = s.match(/^(\d{4})-(\d{2})-(\d{2})(?:[
T](\d{2}):(\d{2})(?::(\d{2}))?)?$/);
 if (m) return new Date(+m[1], +m[2]-1, +m[3], +(m[4]||0), +(m[5]||0),
+(m[6]||0));

 m = s.match(/^(\d{4})\/(\d{2})\/(\d{2})$/);
 if (m) return new Date(+m[1], +m[2]-1, +m[3]);

 m =
s.match(/^(\d{2})\/(\d{2})\/(\d{4})(?:\s+(\d{2}):(\d{2})(?::(\d{2}))?)?$/);
 if (m) return new Date(+m[3], +m[2]-1, +m[1], +(m[4]||0), +(m[5]||0),
+(m[6]||0));

 const fb = new Date(s);
 return isNaN(fb) ? null : fb;
}

function formatDate_(d) {
 const pad = n => String(n).padStart(2, '0');
 return `${d.getFullYear()}-${pad(d.getMonth()+1)}-${pad(d.getDate())} ` +
```

```
`${pad(d.getHours())}:${pad(d.getMinutes())}:${pad(d.getSeconds())}`;
}

function postToSlack_(text) {
 if (!CONFIG.SLACK_WEBHOOK || CONFIG.SLACK_WEBHOOK.includes('REPLACE')) {
   Logger.log('Slack webhook not configured — message would have been:\n' +
text);
   return;
 }
 UrlFetchApp.fetch(CONFIG.SLACK_WEBHOOK, {
   method: 'post',
   contentType: 'application/json',
   payload: JSON.stringify({ text }),
   muteHttpExceptions: true,
 });
}
```

Screenshots from the Slack